`env.action_space` object to sample a random action. Soon enough, we'll sample actions from a trained policy network that will act as our reinforcement learning agent.

Once we initialize the environment, we are free to interact with it via the `step` method. The `step` method returns four important variables that our training loop needs access to in order to run. The first parameter, `state`, represents the next state after we take the action. The second parameter, `reward`, is the reward at that time step, which for our CartPole problem is 1 unless the pole has fallen down. The third parameter, `done`, is a Boolean that indicates whether or not a terminal state has been reached. For our CartPole problem, this would always return `false` until the pole has fallen or the cart has moved outside the window. The last parameter, `info`, is a dictionary with diagnostic information that may be useful for debugging, but we will not use it. That's all you need to know to get most environments up and running in OpenAI Gym.

4.4. The REINFORCE algorithm

Now that you how to create an OpenAI Gym environment and have hopefully developed an intuition for the policy gradient algorithm, let's dive in to getting a working implementation. Our discussion of policy gradients in the previous section focused on a particular algorithm that has been around for decades (like most of deep learning and reinforcement learning) called REINFORCE (yes, it's always fully capitalized). We're going to consolidate what we discussed previously, formalize it, and then turn it into Python code. Let's implement the REINFORCE algorithm for the CartPole example.

4.4.1. Creating the policy network

We will build and initialize a neural network that serves as a policy network. The policy network will accept state vectors as inputs, and it will produce a (discrete) probability distribution over the possible actions. You can think of the agent as a thin wrapper around the policy network that